МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем автоматизованого проектування



ЗВІТ

до лабораторної роботи №2

з дисципліни «Розпізнавання образів і комп'ютерний зір»

на тему: «Фільтрація, шумопригнічення та виділення контурів цифрових зображень»

Варіант № 1

Виконала:

Ст. гр.ПП-44

Бенькалович М. М.

Прийняв:

асистент каф. САП

Назаровець Т.Б.

Львів — 2026

1. **Тема**

Фільтрація, шумопригнічення та виділення контурів цифрових зображень.

2. Мета роботи

Ознайомитися з математичними основами та практичними алгоритмами просторової фільтрації цифрових зображень; дослідити вплив імпульсного шуму «сіль та перець» на зображення технічної деталі; навчитися обчислювати кількісні метрики якості шумопригнічення (MSE, PSNR, SSIM); порівняти лінійний (Box) та нелінійний (Median) фільтри; опанувати оператори виділення контурів Собеля, Лапласа та детектор Canny засобами Python і OpenCV.

3. Індивідуальне завдання (варіант 1)

Таблиця 1. Параметри індивідуального завдання

Параметр	**Значення**
Тип шуму	Salt-and-Pepper, щільність 5 %
Метод фільтрації	Median Filter (для порівняння — Box Filter)
Розмір ядра	3×3, 5×5
Пороги Canny	T_low = 30, T_high = 100
Спеціальне дослідження	Порівняти MSE для Median та Box при однакових апертурах (3×3, 5×5); описати математичну причину стійкості медіани до імпульсного шуму

4. Теоретичні відомості

4.1. Імпульсний шум «сіль та перець»

Імпульсний шум виникає через збої сенсора, помилки передавання чи АЦП. Частина пікселів з імовірністю p незалежно замінюється на крайні значення: половина — на 0 («перець»), половина — на 255 («сіль»). При p = 5 % у зображенні 512×512 спотворюється близько 13 тис. пікселів. Особливість такого шуму — величезна амплітуда окремих викидів при незмінних сусідніх пікселях.

4.2. Усереднювальний (Box) фільтр

Box-фільтр — лінійна згортка з ядром, усі коефіцієнти якого дорівнюють 1/N, де $N = (2k+1)^2$. Вихідний піксель — середнє арифметичне значень у вікні:

$$O(x, y) = (1/N) \cdot \Sigma\, I(x+m, y+n), \quad m, n \in [-k, k]$$

Будь-який імпульс входить у середнє з вагою 1/N і зсуває результат на $(255 - I)/N$. Для вікна 3×3 одиночна «сіль» на фоні 150 зсуває середнє приблизно на 12 рівнів, тобто імпульс не видаляється, а «розмазується» у пляму розміром з апертуру.

4.3. Медіанний фільтр

Медіанний фільтр — нелінійний порядковий фільтр: значення пікселів у вікні Ω впорядковуються, і на вихід подається середній (медіанний) елемент:

$$I_median(x, y) = med\{ I(x+m, y+n) \mid (m, n) \in \Omega \}$$

Медіана є розв'язком задачі мінімізації суми абсолютних відхилень $\Sigma|x_i - m|$, тоді як середнє мінімізує суму квадратів $\Sigma(x_i - m)^2$. Квадратична функція втрат дає великим викидам квадратично велику вагу, а модульна — лише лінійну, і результат залежить від порядку значень, а не від їхньої величини.

4.4. Метрики якості

$$MSE = (1 / (H \cdot W)) \cdot \Sigma\,\Sigma\, (I_orig(y, x) - I_proc(y, x))^2$$

$$PSNR = 10 \cdot \log_{10} (255^2 / MSE), \ dB$$

Чим менше MSE і більше PSNR, тим ближче відфільтроване зображення до еталона. Додатково використано індекс структурної подібності $SSIM \in [-1; 1]$, який враховує яскравість, контраст і структуру (значення понад 0,9 означає високу подібність).

4.5. Виділення контурів

Оператор Собеля апроксимує першу похідну ядрами 3×3 і дає модуль градієнта $G = \sqrt{(Gx^2 + Gy^2)}$. Оператор Лапласа (ядро [[0,1,0],[1,−4,1],[0,1,0]]) — друга похідна, контур відповідає переходу через нуль; він дуже чутливий до шуму. Детектор Canny послідовно виконує гаусове згладжування, обчислення градієнта Собелем, пригнічення немаксимумів (NMS), подвійну порогову обробку (T_low, T_high) та гістерезисне трасування: слабкі пікселі зберігаються лише за наявності 8-зв'язку з сильними.

5. Програмний код

Програма реалізована мовою Python 3 з використанням бібліотек OpenCV, NumPy, Matplotlib та scikit-image (для SSIM). Вхідне зображення — фото деталі detail.jpg (за його відсутності генерується синтетична технічна деталь — фланець з шістьма кріпильними отворами, шпонковим пазом і тонкими рисками маркування, 512×512 пікселів). Для відтворюваності шуму використано фіксований seed = 42. Повний лістинг з коментарями наведено нижче.

GitHub

```
import os
import sys
import json
import time

import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")                    # рендер без вікна (для збереження рисунків)
import matplotlib.pyplot as plt
from skimage.metrics import structural_similarity as ssim

# ---------------------------------------------------------------------------
# Параметри варіанту 1
# ---------------------------------------------------------------------------
NOISE_PROB = 0.05          # щільність імпульсного шуму 5 %
KSIZES = (3, 5)            # апертури фільтрів
T_LOW, T_HIGH = 30, 100    # пороги детектора Canny
SEED = 42                  # фіксований seed для відтворюваності результатів
OUT = "results"
os.makedirs(OUT, exist_ok=True)
plt.rcParams.update({"font.size": 10, "figure.dpi": 110})


# ---------------------------------------------------------------------------
# 1. Завантаження вхідного зображення технічної деталі
# ---------------------------------------------------------------------------
def load_image(path):
    """Завантажує зображення у градаціях сірого.
    Якщо файл не знайдено або його не вдалося прочитати — повідомляє користувача і завершує
роботу."""
    if not os.path.exists(path):
        sys.exit(f"Помилка: файл зображення '{path}' не знайдено. "
                 f"Покладіть зображення поруч зі скриптом або вкажіть шлях: "
                 f"python lab2_variant1.py <шлях_до_зображення>")
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        sys.exit(f"Помилка: не вдалося прочитати '{path}' як зображення "
                 f"(підтримуються формати JPG, PNG, BMP тощо).")
    print(f"Завантажено зображення: {path}  {img.shape}")
    return img


# ---------------------------------------------------------------------------
# 2. Моделювання імпульсного шуму «сіль та перець»
# ---------------------------------------------------------------------------
def add_salt_pepper(img, prob, seed=SEED):
    """prob/2 пікселів -> 0 (перець), prob/2 пікселів -> 255 (сіль)."""
    rng = np.random.default_rng(seed)
    noisy = img.copy()
    rnd = rng.random(img.shape)
    noisy[rnd < prob / 2] = 0
    noisy[rnd > 1 - prob / 2] = 255
    return noisy
```

```
# ---------------------------------------------------------------------------
# 3. Метрики якості
# ---------------------------------------------------------------------------
def calc_metrics(orig, proc):
    """Повертає MSE, PSNR (dB) та SSIM між еталоном і обробленим зображенням."""
    diff = orig.astype(np.float64) - proc.astype(np.float64)
    mse = float(np.mean(diff ** 2))
    psnr = float("inf") if mse == 0 else 10 * np.log10(255.0 ** 2 / mse)
    return mse, psnr, float(ssim(orig, proc, data_range=255))


def timed(fn, *args, repeats=20):
    """Середній час виконання фільтра, мс."""
    fn(*args)
    t0 = time.perf_counter()
    for _ in range(repeats):
        res = fn(*args)
    return res, (time.perf_counter() - t0) / repeats * 1000


# ---------------------------------------------------------------------------
# ОСНОВНИЙ ЕКСПЕРИМЕНТ
# ---------------------------------------------------------------------------
img = load_image(sys.argv[1] if len(sys.argv) > 1 else "detail.png")
cv2.imwrite(f"{OUT}/00_original.png", img)

noisy = add_salt_pepper(img, NOISE_PROB)
cv2.imwrite(f"{OUT}/01_noisy_sp5.png", noisy)
n_imp = int(np.sum(noisy != img))
print(f"Змінено імпульсами пікселів: {n_imp} ({n_imp / img.size * 100:.2f} %)")

# --- фільтрація: Median та Box з однаковими апертурами ---
results = {"Без фільтра": noisy}
times = {}
for k in KSIZES:
    results[f"Median {k}×{k}"], times[f"Median {k}×{k}"] = timed(cv2.medianBlur, noisy, k)
    results[f"Box {k}×{k}"], times[f"Box {k}×{k}"] = timed(cv2.blur, noisy, (k, k))
for name, im in results.items():
    cv2.imwrite(f"{OUT}/f_{name.replace(' ', '_').replace('×', 'x')}.png", im)

# --- метрики ---
table = []
print(f"\n{'Метод':<14}{'MSE':>10}{'PSNR, dB':>10}{'SSIM':>8}{'t, мс':>8}")
for name, im in results.items():
    mse, psnr, s = calc_metrics(img, im)
    row = dict(method=name, mse=round(mse, 2), psnr=round(psnr, 2), ssim=round(s, 4),
               time_ms=round(times.get(name, 0.0), 3))
    table.append(row)
    print(f"{name:<14}{mse:>10.2f}{psnr:>10.2f}{s:>8.4f}{row['time_ms']:>8.3f}")

# виграш медіани над Box
gain = {}
for k in KSIZES:
    m = next(r for r in table if r["method"] == f"Median {k}×{k}")
    b = next(r for r in table if r["method"] == f"Box {k}×{k}")
    gain[k] = dict(mse_ratio=round(b["mse"] / m["mse"], 2), psnr_gain=round(m["psnr"] -
b["psnr"], 2))
    print(f"Апертура {k}×{k}: MSE_Box / MSE_Median = {gain[k]['mse_ratio']}, "
          f"виграш PSNR = {gain[k]['psnr_gain']} dB")

# --- Рис. 1: вхідні дані та результати фільтрації ---
fig, axes = plt.subplots(2, 3, figsize=(12, 9.2))
order = ["Без фільтра", "Median 3×3", "Median 5×5", None, "Box 3×3", "Box 5×5"]
for ax, name in zip(axes.ravel(), order):
    if name is None:
        ax.imshow(img, cmap="gray", vmin=0, vmax=255); ax.set_title("Оригінал (еталон)")
    else:
        r = next(r for r in table if r["method"] == name)
        ttl = "Зашумлене S&P 5 %" if name == "Без фільтра" else name
        ax.imshow(results[name], cmap="gray", vmin=0, vmax=255)
        ax.set_title(f"{ttl}\nMSE={r['mse']:.1f}  PSNR={r['psnr']:.2f} dB")
    ax.axis("off")
fig.tight_layout(); fig.savefig(f"{OUT}/fig1_filtering.png", dpi=130); plt.close(fig)

# --- Рис. 2: збільшений фрагмент (видно залишки імпульсів / розмиття) ---
y0, x0, s = 60, 330, 110      # фрагмент з кріпильним отвором
fig, axes = plt.subplots(1, 5, figsize=(15, 3.4))
for ax, (name, im) in zip(axes, [("Оригінал", img), ("S&P 5 %", noisy),
                                 ("Median 3×3", results["Median 3×3"]),
```

```
                                  ("Median 5×5", results["Median 5×5"]),
                                  ("Box 5×5", results["Box 5×5"])]):
    ax.imshow(im[y0:y0 + s, x0:x0 + s], cmap="gray", vmin=0, vmax=255, interpolation="nearest")
    ax.set_title(name); ax.axis("off")
fig.tight_layout(); fig.savefig(f"{OUT}/fig2_zoom.png", dpi=130); plt.close(fig)

# --- Рис. 3: стовпчикова діаграма MSE (основне спец. дослідження) ---
fig, ax = plt.subplots(figsize=(7, 4))
xs = np.arange(len(KSIZES)); wbar = 0.36
mse_med = [next(r["mse"] for r in table if r["method"] == f"Median {k}×{k}") for k in KSIZES]
mse_box = [next(r["mse"] for r in table if r["method"] == f"Box {k}×{k}") for k in KSIZES]
b1 = ax.bar(xs - wbar / 2, mse_med, wbar, label="Median", color="#2a6fbb")
b2 = ax.bar(xs + wbar / 2, mse_box, wbar, label="Box", color="#e08a2c")
ax.bar_label(b1, fmt="%.1f"); ax.bar_label(b2, fmt="%.1f")
ax.set_xticks(xs, [f"{k}×{k}" for k in KSIZES]); ax.set_ylabel("MSE")
ax.set_title("MSE для Median та Box при однакових апертурах (S&P 5 %)")
ax.legend(); ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout(); fig.savefig(f"{OUT}/fig3_mse_bar.png", dpi=130); plt.close(fig)

# --- Рис. 4: 1D-профіль інтенсивності вздовж рядка через отвір ---
row_y = 256
fig, ax = plt.subplots(figsize=(10, 3.8))
ax.plot(noisy[row_y], color="#bbbbbb", lw=0.8, label="S&P 5 %")
ax.plot(img[row_y], color="black", lw=1.6, label="Оригінал")
ax.plot(results["Median 5×5"][row y], color="#2a6fbb", lw=1.3, label="Median 5×5")
ax.plot(results["Box 5×5"][row_y], color="#e08a2c", lw=1.3, ls="--", label="Box 5×5")
ax.set_xlim(0, img.shape[1] - 1); ax.set_xlabel("x, піксель"); ax.set_ylabel("Інтенсивність")
ax.set_title(f"Профіль інтенсивності вздовж рядка y = {row_y}")
ax.legend(ncol=4, loc="upper center", bbox_to_anchor=(0.5, -0.2), frameon=False);
ax.spines[["top", "right"]].set_visible(False)
fig.tight layout(); fig.savefig(f"{OUT}/fig4 profile.png", dpi=130); plt.close(fig)

# ---------------------------------------------------------------------------
# 4. Математична причина стійкості медіани: аналіз одного вікна 3×3 зі спайком
# ---------------------------------------------------------------------------
win = np.array([[150, 152, 149], [151, 255, 150], [148, 153, 151]], dtype=float)  # «сіль» у
центрі
win_mean, win_med = win.mean(), np.median(win)
print(f"\nВікно 3×3 зі спайком 255: mean = {win_mean:.2f}, median = {win_med:.0f}")
# скільки імпульсів має потрапити у вікно, щоб медіана «зламалась»
# медіана 9 значень — 5-те за порядком; вона стає імпульсом, коли однакових імпульсів ≥ 5
breakdown = {k: (k * k) // 2 + 1 for k in (3, 5, 7)}
# ймовірність, що у вікні ≥ m імпульсів одного знаку (p/2 = 2.5 %) — біноміальний розподіл
from math import comb
def p_break(k, p=NOISE_PROB / 2):
    n = k * k; m = n // 2 + 1
    return sum(comb(n, j) * p ** j * (1 - p) ** (n - j) for j in range(m, n + 1))
p_fail = {k: p_break(k) for k in (3, 5)}
for k in (3, 5):
    print(f"P(медіана {k}×{k} дорівнює імпульсу) = {p_fail[k]:.2e}")

# Емпірична перевірка: частка імпульсів, що залишились після фільтра
imp_mask = noisy != img
residual_imp = {}
for name in ["Median 3×3", "Median 5×5", "Box 3×3", "Box 5×5"]:
    f = results[name].astype(int)
    # «залишковий» імпульс — піксель, відхилення якого від еталону > 40 рівнів
    residual_imp[name] = int(np.sum(np.abs(f - img.astype(int)) > 40))
print("Пікселів з похибкою > 40:", residual_imp)

# ---------------------------------------------------------------------------
# 5. Додатково: залежність MSE від щільності шуму (1…30 %) — межі ефективності
# ---------------------------------------------------------------------------
densities = [0.01, 0.03, 0.05, 0.08, 0.10, 0.15, 0.20, 0.25, 0.30]
sweep = {n: [] for n in ["Median 3×3", "Median 5×5", "Box 3×3", "Box 5×5"]}
for d in densities:
    nz = add_salt_pepper(img, d)
    for k in KSIZES:
        sweep[f"Median {k}×{k}"].append(calc_metrics(img, cv2.medianBlur(nz, k))[0])
        sweep[f"Box {k}×{k}"].append(calc_metrics(img, cv2.blur(nz, (k, k)))[0])
fig, ax = plt.subplots(figsize=(8, 4.2))
sty = {"Median 3×3": ("#7fb3e6", "-"), "Median 5×5": ("#2a6fbb", "-"),
       "Box 3×3": ("#f2b878", "--"), "Box 5×5": ("#e08a2c", "--")}
for n, v in sweep.items():
    ax.plot(np.array(densities) * 100, v, marker="o", ms=4, color=sty[n][0], ls=sty[n][1],
label=n)
ax.axvline(5, color="gray", lw=0.8, ls=":"); ax.text(5.3, ax.get_ylim()[1] * 0.92, "варіант 1",
color="gray")
ax.set_yscale("log"); ax.set_xlabel("Щільність шуму S&P, %"); ax.set_ylabel("MSE (лог. шкала)")
ax.set_title("Залежність MSE від щільності імпульсного шуму")
```

```
ax.legend(); ax.grid(alpha=0.3, which="both"); ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout(); fig.savefig(f"{OUT}/fig5_density_sweep.png", dpi=130); plt.close(fig)

# ---------------------------------------------------------------------------
# 6. Виділення контурів: Sobel, Laplacian, Canny (30/100)
# ---------------------------------------------------------------------------
base = results["Median 5×5"]
sobelx = cv2.Sobel(base, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(base, cv2.CV_64F, 0, 1, ksize=3)
sobel_mag = cv2.magnitude(sobelx, sobely)
laplacian = cv2.Laplacian(base, cv2.CV_64F, ksize=3)

edges_ref = cv2.Canny(img, T_LOW, T_HIGH)          # «еталонна» карта контурів оригіналу
edge_stats = []
canny_maps = {}
for name in ["Без фільтра", "Median 3×3", "Median 5×5", "Box 3×3", "Box 5×5"]:
    e = cv2.Canny(results[name], T_LOW, T_HIGH)
    canny_maps[name] = e
    # точність / повнота з допуском 1 піксель (розширюємо еталон та результат)
    k3 = np.ones((3, 3), np.uint8)
    tp_p = np.sum((e > 0) & (cv2.dilate(edges_ref, k3) > 0))
    tp_r = np.sum((edges_ref > 0) & (cv2.dilate(e, k3) > 0))
    prec = tp_p / max(np.sum(e > 0), 1); rec = tp_r / max(np.sum(edges_ref > 0), 1)
    f1 = 2 * prec * rec / (prec + rec + 1e-12)
    edge_stats.append(dict(method=name, n_edge=int(np.sum(e > 0)),
                           precision=round(float(prec), 3), recall=round(float(rec), 3),
                           f1=round(float(f1), 3)))
n_ref = int(np.sum(edges_ref > 0))
print(f"\nCanny {T_LOW}/{T_HIGH}: еталон (оригінал) N_edge = {n_ref}")
for r in edge_stats:
    print(f"  {r['method']:<12} N edge={r['n_edge']:>6}  P={r['precision']:.3f}  "
          f"R={r['recall']:.3f}  F1={r['f1']:.3f}")

fig, axes = plt.subplots(2, 3, figsize=(12, 9.2))
axes[0, 0].imshow(base, cmap="gray"); axes[0, 0].set_title("Median 5×5 (вхід для контурів)")
axes[0, 1].imshow(sobel_mag, cmap="gray"); axes[0, 1].set_title("Модуль градієнта Собеля |G|")
axes[0, 2].imshow(np.abs(laplacian), cmap="gray", vmax=np.percentile(np.abs(laplacian), 99.5))
axes[0, 2].set_title("|Laplacian|")
axes[1, 0].imshow(canny_maps["Без фільтра"], cmap="gray")
axes[1, 0].set_title(f"Canny {T_LOW}/{T_HIGH} без фільтра\nN={edge_stats[0]['n_edge']}")
axes[1, 1].imshow(canny_maps["Median 5×5"], cmap="gray")
axes[1, 1].set_title(f"Canny {T_LOW}/{T_HIGH} після Median 5×5\nN={edge_stats[2]['n_edge']}")
axes[1, 2].imshow(canny_maps["Box 5×5"], cmap="gray")
axes[1, 2].set_title(f"Canny {T_LOW}/{T_HIGH} після Box 5×5\nN={edge_stats[4]['n_edge']}")
for ax in axes.ravel(): ax.axis("off")
fig.tight_layout(); fig.savefig(f"{OUT}/fig6_edges.png", dpi=130); plt.close(fig)

# ---------------------------------------------------------------------------
# 7. Вплив попереднього Gaussian Blur на Canny (пункт порядку виконання роботи)
# ---------------------------------------------------------------------------
blur_study = []
for sigma in [0, 0.8, 1.4, 2.0, 3.0]:
    src = results["Median 3×3"] if sigma == 0 else cv2.GaussianBlur(results["Median 3×3"], (0,
0), sigma)
    e = cv2.Canny(src, T_LOW, T_HIGH)
    k3 = np.ones((3, 3), np.uint8)
    prec = np.sum((e > 0) & (cv2.dilate(edges_ref, k3) > 0)) / max(np.sum(e > 0), 1)
    rec = np.sum((edges_ref > 0) & (cv2.dilate(e, k3) > 0)) / max(n_ref, 1)
    blur_study.append(dict(sigma=sigma, n_edge=int(np.sum(e > 0)), precision=round(float(prec),
3),
                           recall=round(float(rec), 3), f1=round(float(2 * prec * rec / (prec +
rec + 1e-12)), 3)))
print("\nGaussian Blur перед Canny (на Median 3×3):")
for r in blur_study:
    print(f"  σ={r['sigma']:<4} N={r['n_edge']:>6} P={r['precision']:.3f} R={r['recall']:.3f}
F1={r['f1']:.3f}")

# ---------------------------------------------------------------------------
# Збереження числових результатів
# ---------------------------------------------------------------------------
summary = dict(image_shape=list(img.shape), impulses=n_imp, metrics=table, gain=gain,
               window=dict(values=win.astype(int).tolist(), mean=round(win_mean, 2),
median=float(win_med)),
               breakdown=breakdown, p_fail={k: float(v) for k, v in p_fail.items()},
               residual_imp=residual_imp, sweep=dict(densities=densities,
               **{k: [round(x, 2) for x in v] for k, v in sweep.items()}),
               canny_ref=n_ref, edges=edge_stats, blur_study=blur_study)
with open(f"{OUT}/summary.json", "w", encoding="utf-8") as fjs:
    json.dump(summary, fjs, ensure_ascii=False, indent=2)
print(f"\nРезультати збережено у папці '{OUT}/'.")
```

6. Результати

6.1. Початкові дані та зашумлення

Вхідне зображення — напівтонове (Grayscale) 512×512. Після моделювання шуму «сіль та перець» з p = 5 % змінено 13073 пікселів (4,99 %), що відповідає заданій щільності. MSE зашумленого кадру відносно еталона — 922,23, PSNR — 18,48 dB, SSIM — 0,242.

6.2. Результати фільтрації (проміжні та кінцеві)

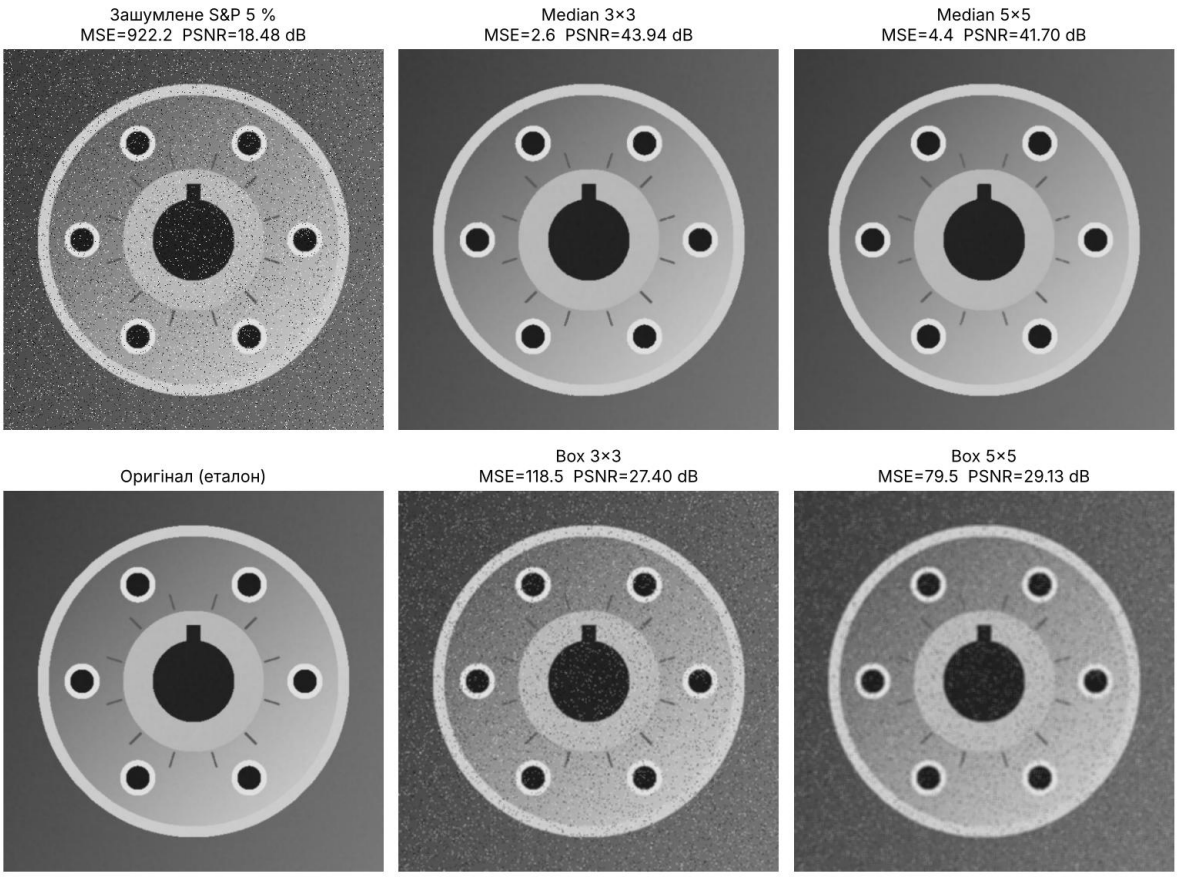


Рис. 1. Оригінал, зашумлене зображення та результати фільтрації Median і Box (3×3, 5×5)

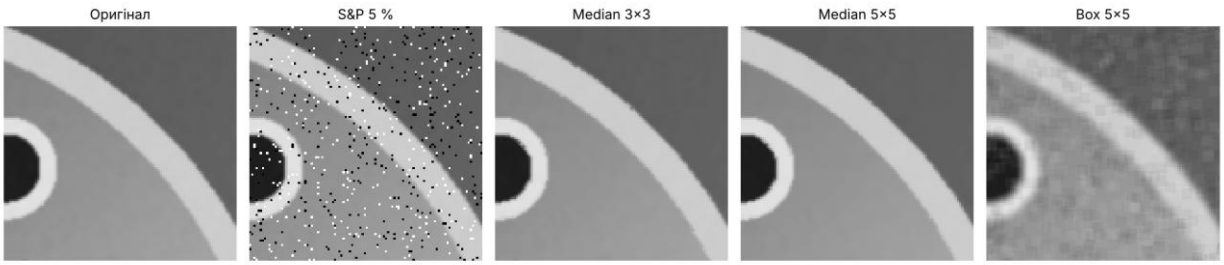


Рис. 2. Збільшений фрагмент із кріпильним отвором (110×110 пікселів)

Таблиця 2. Метрики якості шумопригнічення (S&P 5 %)

Метод	MSE	PSNR, dB	SSIM	Час, мс
Без фільтра (зашумлене)	922,23	18,48	0,2419	—
Median 3×3	2,62	43,94	0,9965	0,069
Box 3×3	118,46	27,40	0,5344	0,049
Median 5×5	4,39	41,70	0,9927	0,514
Box 5×5	79,53	29,13	0,7358	0,064

Час виконання виміряно як середнє за 20 повторів; він залежить від процесора і наведений лише для порівняння між фільтрами.

6.3. Спеціальне дослідження: MSE для Median та Box

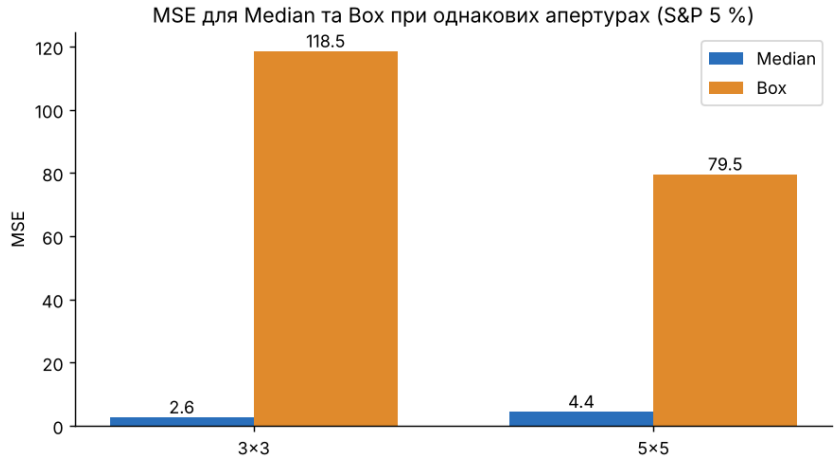


Рис. 3. MSE фільтрів Median та Box при однакових апертурах

Таблиця 3. Порівняння Median і Box при однакових апертурах

Апертура	MSE Median	MSE Box	MSE_Box / MSE_Median	Виграш PSNR, dB
3×3	2,62	118,46	45,21	16,54
5×5	4,39	79,53	18,12	12,57

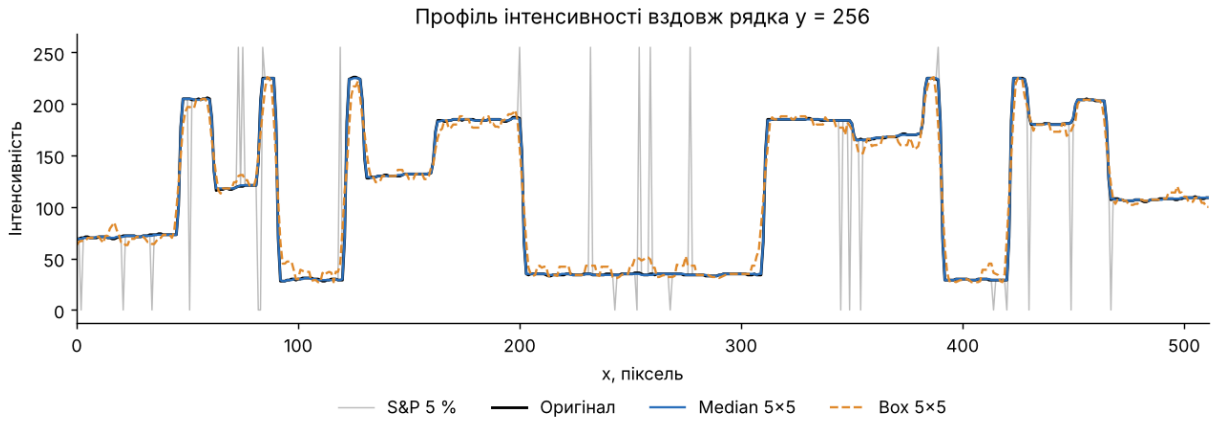


Рис. 4. Профіль інтенсивності вздовж рядка y = 256 (через центр деталі)

Аналіз окремого вікна. Розглянемо вікно 3×3 з «сіллю» в центрі: [150, 152, 149; 151, 255, 150; 148, 153, 151]. Середнє арифметичне (Box) = 162,11, тобто похибка ≈ 11 рівнів. Упорядкований ряд: 148, 149, 150, 150, 151, 151, 152, 153, 255 — медіана (5-й елемент) = 151, що збігається з істинним рівнем фону. Імпульс просто опиняється на краю впорядкованого ряду і на результат не впливає.

Ймовірність «пробою» медіани. Щоб медіана вікна з $N = k^2$ пікселів стала імпульсом, у вікні має опинитися щонайменше $\lfloor N/2 \rfloor + 1$ однакових імпульсів (5 для 3×3, 13 для 5×5). При ймовірності кожного типу імпульсу p/2 = 2,5 % біноміальний розподіл дає P ≈ 1,13e-6 для 3×3 та P ≈ 5,85e-15 для 5×5 — практично нуль. Емпірично кількість пікселів з похибкою понад 40 рівнів після фільтрації становить: Median 3×3 — 87, Median 5×5 — 147, Box 3×3 — 1860, Box 5×5 — 954 (у медіанних фільтрів це переважно пікселі на межах, а не залишкові імпульси).

6.4. Додаткове дослідження: вплив щільності шуму

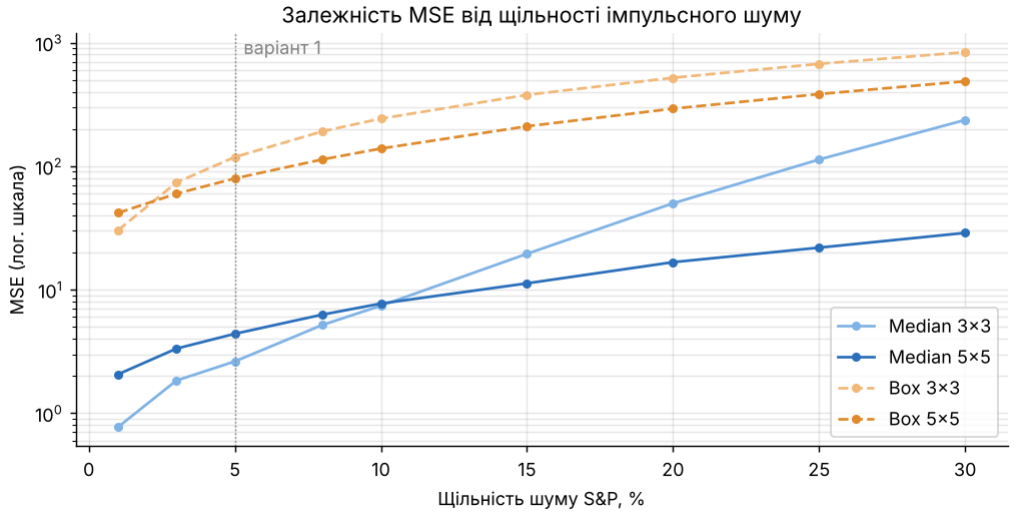


Рис. 5. Залежність MSE від щільності шуму «сіль та перець» (1–30 %)

Таблиця 4. MSE при різній щільності шуму

Щільність	Median 3×3	Median 5×5	Box 3×3	Box 5×5
1 %	0,77	2,06	30,25	41,81
3 %	1,84	3,33	73,75	59,80
5 %	2,62	4,39	118,46	79,53
8 %	5,18	6,28	191,25	113,36
10 %	7,41	7,72	242,58	138,80
15 %	19,51	11,25	377,48	210,19
20 %	50,02	16,69	519,83	291,94
25 %	113,16	21,89	673,12	383,13
30 %	236,42	28,81	837,13	486,27

6.5. Виділення контурів: Sobel, Laplacian, Canny (30/100)

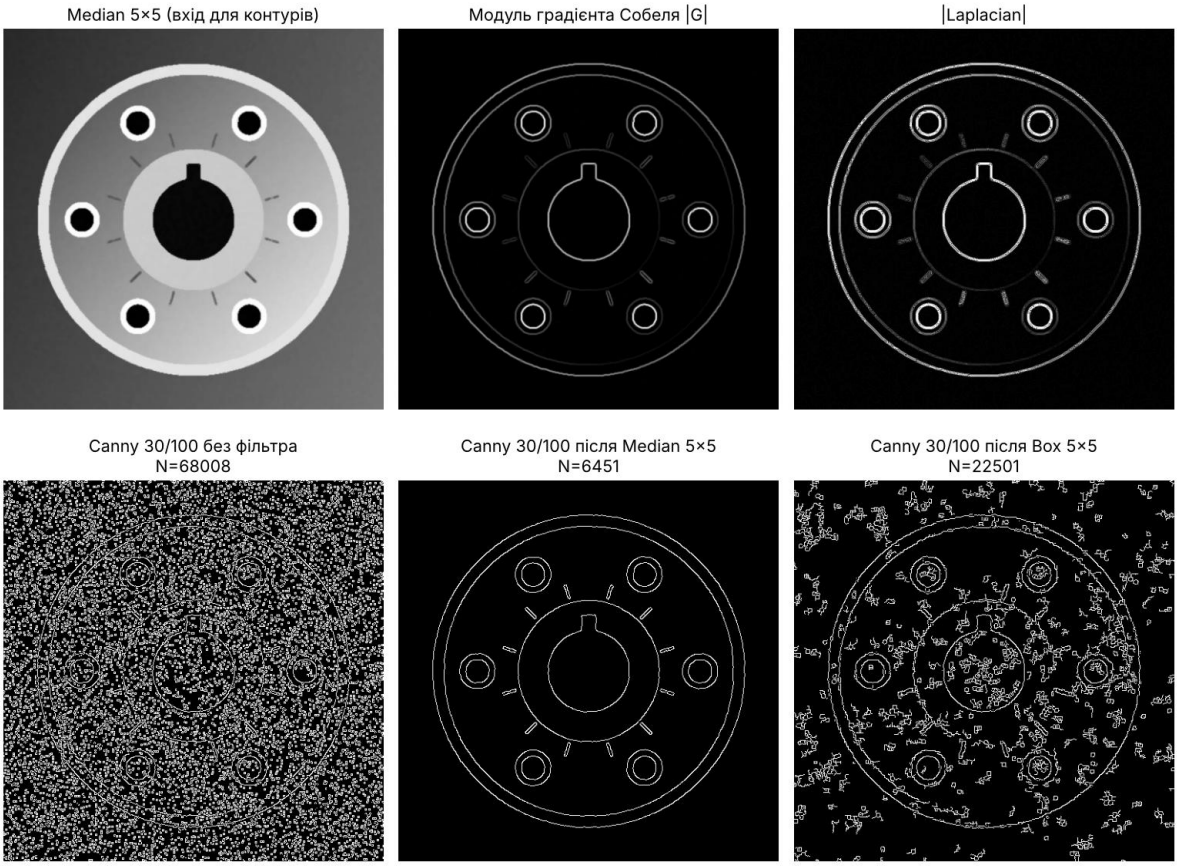


Рис. 6. Оператори Собеля, Лапласа та детектор Canny 30/100 до і після фільтрації

Як еталон взято карту Canny 30/100 для незашумленого оригіналу (N_edge = 6436). Для кожного варіанта обчислено кількість контурних пікселів N_edge, точність (precision), повноту (recall) та F-міру з допуском 1 піксель.

Таблиця 5. Якість контурів Canny 30/100

Вхід для Canny	**N_edge**	**Precision**	**Recall**	**F1**
Без фільтра	68008	0,110	0,995	0,199
Median 3×3	6485	1,000	1,000	1,000
Median 5×5	6451	1,000	0,997	0,999
Box 3×3	57764	0,118	0,978	0,211
Box 5×5	22501	0,293	0,930	0,446

Дослідження впливу попереднього Gaussian Blur на Canny (застосовано після Median 3×3):

Таблиця 6. Вплив σ гаусового розмиття на Canny 30/100

σ	**N_edge**	**Precision**	**Recall**	**F1**
0 (без розмиття)	6485	1,000	1,000	1,000

σ	N_edge	Precision	Recall	F1
0,8	6214	1,000	0,962	0,981
1,4	5722	1,000	0,888	0,941
2,0	5346	1,000	0,836	0,911
3,0	3784	1,000	0,610	0,758

7. Аналіз результатів

Що було зроблено. Змодельовано імпульсний шум 5 % на зображенні технічної деталі, виконано фільтрацію медіанним та усереднювальним фільтрами з апертурами 3×3 і 5×5, обчислено MSE, PSNR і SSIM, побудовано профіль інтенсивності, проаналізовано поведінку фільтрів у межах одного вікна та в діапазоні щільностей шуму 1–30 %, виділено контури операторами Собеля, Лапласа та детектором Canny 30/100 і кількісно оцінено якість контурів.

Які параметри використовувались. p = 5 % (seed = 42); cv2.medianBlur з ksize = 3, 5; cv2.blur з ядрами (3, 3), (5, 5); Sobel ksize = 3; Laplacian ksize = 3; Canny T_low = 30, T_high = 100; σ гаусового розмиття перед Canny — 0…3.

Чому отримано такий результат. Медіанний фільтр зменшив MSE з 922,23 до 2,62 (3×3), тоді як Box — лише до 118,46. При однакових апертурах MSE у Box більше у 45,21 раза (3×3) та у 18,12 раза (5×5), виграш медіани за PSNR — 16,54 та 12,57 dB. Причина — математична природа медіани: вона є порядковою статистикою з точкою пробою 50 %, тобто ігнорує величину викидів, доки їх менше половини вікна. Середнє ж має точку пробою 0 %: навіть один імпульс зсуває результат пропорційно своїй амплітуді. Тому Box не видаляє імпульси, а перетворює їх на сірі плями (рис. 1, 2), і одночасно розмиває межі деталі (рис. 4). Цікаво, що для Box більша апертура 5×5 дає менше MSE (імпульс розподіляється на 25 пікселів, а не на 9), а для медіани — навпаки: 3×3 вже повністю прибирає шум 5 %, а 5×5 лише сильніше «зрізає» кути та тонкі риски маркування.

Для контурів результат ще наочніший: на зашумленому кадрі Canny знаходить 68008 контурних пікселів (у 10,6 раза більше, ніж на еталоні; precision 0,11), бо кожен імпульс дає сильний градієнт, вищий за T_high = 100. Після медіани F1 ≈ 1,000, після Box 5×5 — лише 0,446: розмиті плями від імпульсів усе ще мають градієнт понад низький поріг T_low = 30. Додаткове

гаусове розмиття після медіани в цьому випадку не потрібне: precision уже 1,0, а з ростом σ лише падає recall (втрачаються тонкі риски).

Які виникли помилки та обмеження. Синтетичне зображення має кусково-гладку структуру без дрібної текстури, тому медіана працює майже ідеально (SSIM > 0,99); на реальному фото з текстурою металу похибка медіани буде більшою. На першій версії рисунків підписи другого ряду накладалися на зображення — виправлено збільшенням висоти фігури. Зі збільшенням щільності шуму понад ≈ 10 % медіана 3×3 стає гіршою за 5×5 (рис. 5), бо у вікні з 9 пікселів частіше набирається 5 імпульсів.

Як можна покращити алгоритм. Використати адаптивний медіанний фільтр, який замінює лише пікселі-імпульси (значення z_min або z_max вікна) і збільшує апертуру тільки за потреби — це зберігає дрібні деталі та стійкий до шуму 20–30 %; застосувати детектор імпульсів (наприклад, за значеннями 0/255) і фільтрувати лише позначені пікселі; для Canny підбирати пороги автоматично (метод Оцу або медіана градієнта).

8. Висновки

1. Для імпульсного шуму «сіль та перець» 5 % медіанний фільтр значно ефективніший за усереднювальний: при однакових апертурах MSE менше у 45 (3×3) та 18 (5×5) разів, PSNR вищий на 16,5 та 12,6 dB відповідно.

2. Оптимальна апертура для варіанта 1 — Median 3×3 (PSNR 43,94 dB, SSIM 0,997). Апертура 5×5 доцільна лише при щільності шуму понад ≈ 10 %.

3. Стійкість медіани пояснюється тим, що вона мінімізує суму модулів відхилень і є порядковою статистикою з точкою пробою 50 %: імпульси опиняються на краях упорядкованого ряду і не впливають на результат, доки їх менше половини вікна. Ймовірність такої події при 5 % шуму — порядку 10^{-6} для 3×3.

4. Без попередньої фільтрації детектор Canny 30/100 непридатний (F1 = 0,20); після медіанного фільтра контури деталі відновлюються майже повністю (F1 ≈ 1,00), тоді як лінійне розмиття Box не усуває фальшивих контурів.

9. Відповіді на контрольні питання

1. Двовимірна згортка — це зважена сума значень пікселів у ковзному вікні: $O(x,y) = \Sigma\,\Sigma\, I(x+m, y+n)\cdot K(m+k, n+k)$. Ядро K визначає вагу кожного сусіда; операція лінійна, комутативна та асоціативна, а для збереження середньої яскравості сума коефіцієнтів ядра має дорівнювати 1.
2. Box-фільтр надає всім пікселям вікна однакову вагу 1/N, тому сильніше розмиває межі й дає «дзвонові» артефакти (його спектр — sinc-функція). Фільтр Гаусса зменшує вагу з відстанню від центру, тож розмиває м'якше та природніше, краще зберігаючи геометрію деталі.
3. Лінійний фільтр усереднює імпульс разом із сусідами, тож його амплітуда 0 або 255 зсуває результат. Медіана — порядкова статистика: одиночний імпульс опиняється на краю впорядкованого ряду і майже ніколи не стає медіаною, тому повністю видаляється без розмиття меж.
4. Просторова вага $g_s(\|p - q\|)$ — гаусіана від геометричної відстані між пікселями, і фотометрична (діапазонна) вага $g_r(|I(p) - I(q)|)$ — гаусіана від різниці інтенсивностей. Їхній добуток не дає усереднювати пікселі з різних боків межі.
5. MSE — середнє квадратів різниць пікселів еталона й обробленого зображення; $PSNR = 10\cdot\log_{10}(255^2/MSE)$ dB. Для якісно відновлених технічних зображень типові PSNR > 30–35 dB (MSE < ≈ 20–65) та SSIM > 0,9.
6. Оператор Собеля обчислює першу похідну згорткою з ядрами 3×3, що поєднують диференціювання в одному напрямку та згладжування [1, 2, 1] в іншому. Gx = [[−1,0,1],[−2,0,2],[−1,0,1]] виявляє вертикальні межі (зміна вздовж x), Gy — транспоноване ядро, виявляє горизонтальні межі.
7. Лапласіан — сума других похідних, що характеризує кривизну функції інтенсивності; межа відповідає переходу через нуль. Диференціювання підсилює високі частоти (у частотній області множення на ω^2), тому шум, який є високочастотним, підсилюється найсильніше.
8. Етапи Canny: 1) гаусове згладжування; 2) обчислення модуля та напрямку градієнта (Собель); 3) пригнічення немаксимумів (NMS); 4) подвійна порогова обробка та гістерезисне трасування контурів.

9. T_high визначає «сильні» пікселі, з яких починаються контури: що він вищий, то менше фальшивих контурів, але більше розривів. T_low визначає, які слабкі пікселі можна приєднати до сильних: занижений T_low додає шумові гілки, завищений — розриває контури. Зазвичай T_high : T_low ≈ 2 : 1 – 3 : 1.

1. Диференціальні оператори підсилюють шум, і кожен шумовий піксель дає хибний градієнт. Попереднє розмиття Гауссом пригнічує високочастотний шум, зменшує кількість фальшивих контурів і стабілізує напрямок градієнта для NMS (для імпульсного шуму замість Гаусса ефективніший медіанний фільтр).